Cloud Computing Mini Project

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

STUDENT	ROLL NUMBER	COURSE
Ankit Yadav	230101069	Cloud Computing (CS3081)

Repository: github.com/ankityadav010101/Cloud_Command_Parser

Project summary

This project builds a constrained natural-language command parser that maps one short English command to a four-key JSON object. The implementation combines local teacher labeling, sequence-level student fine-tuning, an S3-compatible model store, a containerized API, and a local Kubernetes deployment. It uses only the supplied command pool for training; the supplied validation set is reserved for evaluation.

Evidence status: Source code and configuration are prepared. The Ollama labeling run, model training and scoring, container execution, Kubernetes reliability exercises, and Locust load test have not been run in the available environment. Their results are marked pending throughout this report. Replace pending items with measured outputs before final academic submission.

Architecture

3,000 command pool -> parallel Ollama Phi-3.5 workers -> validated and checksummed teacher dataset -> CPU fine-tuned Pythia-70M -> model artifact in SeaweedFS -> FastAPI container -> k3d Ingress and Kubernetes Deployment. Validation rows are used only by the teacher/base/student evaluators.

1. Task 1 - Object storage and parallel teacher labeling

The worker assigns each zero-based source line to one shard using line_number modulo SHARD_COUNT. The fixed Ollama phi3.5 teacher is called with temperature 0 and seed 42. Each raw response is saved with its source ID and text. The prompt requests exactly intent, date, time, and place. Two label workers are configured in a StatefulSet in the pipeline namespace.

The cleaner validates JSON, exact keys, intent membership, and slot text copied from the original command; it removes duplicate IDs/text, orders examples by ID, writes datasets/v1/train.jsonl, and records a SHA-256 checksum. It verifies complete one-time source ID coverage. SeaweedFS provides datasets and models buckets. Validation data is not read by labeling or training code.

Labeling count	Value
Input records	3,000 (provided pool)
Invalid teacher JSON / wrong keys	Pending labeling run
Invalid intent / invalid slot	Pending labeling run
Duplicates / retained records	Pending labeling run
train.jsonl SHA-256	Pending cleaning run

2. Task 2 - Student training with sequence-level distillation

The student base model is EleutherAI/pythia-70m. Training uses CPU PyTorch, Transformers, Datasets, and a LoRA adapter, merged into the saved model directory so the inference service can download a self-contained artifact. Each example contains the command followed by the teacher's exact JSON reply. Configuration is centralized in training/config.yaml: seed 42, up to 1,500 examples, maximum length 192, batch size 2, one epoch, learning rate 0.00005, and weight decay 0.01. Separate v1 and v2 versions are supported.

Sequence-level distillation teaches the student to reproduce the teacher's output sequence for an input command. Logit-based distillation instead matches teacher token probability distributions. It is not implemented because the required Ollama interface returns generated text rather than teacher logits. A limitation is that the student can learn teacher mistakes and inconsistent labels.

Model	JSON validity	Intent	Date	Time	Place	Full frame
Phi-3.5 teacher	Pending	Pending	Pending	Pending	Pending	Pending
Pythia-70M base	Pending	Pending	Pending	Pending	Pending	Pending
Fine-tuned student	Pending	Pending	Pending	Pending	Pending	Pending

Run tools/evaluate_teacher.py and tools/evaluate.py for each model, preserve raw outputs under results/, and use tools/score.py for the six assignment metrics. Never use the validation set to edit teacher labels or training data. Training loss and model scores remain pending until actual runs are complete.

3. Task 3 - Inference service in a container

service/app.py exposes POST /parse, GET /healthz, and GET /readyz. Successful output contains exactly the schema's four keys and includes X-Model-Version and X-Served-By headers. Readiness remains unavailable until model loading completes. At startup the service retrieves model and tokenizer files from S3-compatible storage; credentials are provided through a Kubernetes Secret. FAULT_INJECT=1 exercises the fail-closed HTTP 500 body {"error":"invalid_model_output"}.

The Docker image uses pinned Python dependencies and CPU-only PyTorch; it does not include model weights. The required image-size and 50 sequential-request checks under 1 CPU and 1 GiB have not been run.

Container measurement	Evidence
Image size	Pending image build
50 sequential requests	Pending run
Peak memory	Pending measurement
Fault injection status/body	Pending run

4. Task 4 - Kubernetes deployment, scaling, and reliability

k8s/app.yaml defines the app namespace, two-replica Deployment, rolling update strategy, Service, Ingress, liveness/readiness/startup probes, CPU and memory requests/limits, and HPA targeting 70% CPU with one to two replicas. Readiness prevents traffic from reaching a pod while the model is loading. The k3d setup has one laptop node and cannot demonstrate physical machine fault tolerance.

Reliability exercise	Evidence
HPA scale up/down	Pending run
Roll forward to v2 and rollback to v1	Pending run
Pod recovery	Pending run
Three cold starts and median	Pending run
Traffic per ready replica	Pending load test

5. Task 5 - Subnetting and stateless NACL

The 10.20.0.0/16 block is allocated from largest requirement to smallest. Usable host counts exclude the network and broadcast addresses.

Subnet	Needed	Network	Usable range	Broadcast	Usable
app	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

The deny rule for 198.51.100.0/24 precedes broad HTTP/HTTPS allows because NACLs evaluate lowest rule number first and stop on the first match. NACLs are stateless, so return packets need explicit rules: inbound ephemeral client ports for application replies and outbound ephemeral ports for replies to clients. SSH is limited to management. The sample packet comparison must be captured from make nacl.

6. Task 6 - Reproducibility, tests, and CI/CD

The project includes a Makefile, pinned requirements, central training configuration, four unit checks, and GitHub Actions stages for Python compilation, unit checks, release model download/evaluation, a 95% JSON-validity gate, and secret scanning. The package script generates a SHA-256 sidecar. The clean-clone build, CI workflow, deliberate failed quality gate, and release checksum verification have not been run. Add workflow links and checksum evidence after completing them. AI_USE.md documents AI assistance and human review expectations.

7. Task 7 - Performance, cost, SLO, and review

The prescribed Locust run and kubectl top pods should provide p50/p95 latency, throughput, error rate, and per-pod resources. These measurements are pending. Proposed SLO: 99% of requests succeed during one standard two-minute load-test run. For N total requests the error budget is 0.01 x N failed requests; insert the observed request and failure counts from Locust.

Hypothetical cost calculation

At the manifest limits, one replica costs 1 vCPU x INR 3.50 + 1 GiB x INR 0.45 = INR 3.95 per hour. Two running replicas cost INR 7.90 per hour. Cost per 1,000 requests = INR 7.90 / measured requests per hour x 1,000. The request rate is pending; this is an estimate using the supplied hypothetical rate card, not a cloud charge.

Concern	Evidence	Limitation	Improvement
Data and storage	3,000 input rows; cleaned counts pending	Local single-node store	Record checksum; use replicated storage
Model quality	Validation scores pending	Copies teacher errors	Inspect labels; compare full-frame scores
Latency / throughput	Locust values pending	CPU-only laptop	Profile generation and tokenization
Resource use	Per-pod CPU/RAM pending	Cluster capacity varies	Tune from measured peaks
Reliability	HPA/recovery evidence pending	One physical node	Repeat on multi-node cluster
Security / cost	No cloud charge; estimate above	Hypothetical prices	Use Secrets; cost from measured load

8. Limitations

The 70-million-parameter student is intentionally small and may lose accuracy compared with the fixed teacher. It learns from generated text labels, so teacher errors and ambiguous slot boundaries can propagate. CPU training and inference on a single laptop constrain throughput, and a one-node k3d cluster cannot prove multi-machine fault tolerance. SeaweedFS is a local S3-compatible store for this exercise; it does not provide the durability or availability guarantees of a production object-storage service. The API's generated output quality, startup time, image size, memory use, autoscaling, recovery, and latency remain unverified until the required runs are completed. Therefore, this report does not claim measured model or service performance.

9. Conclusion

The project provides an end-to-end implementation plan and code for teacher-based sequence-level distillation, artifact storage, schema-checked inference, Kubernetes operations, subnetting, NACL simulation, and CI quality gates. It also records the design choices and the evidence required by the assignment. Completion of the engineering implementation does not replace the outstanding experiments: run labeling and training, evaluate all three models, exercise the cluster and service, collect Locust and resource measurements, and update the pending fields before submitting the final report.

References

Bastianelli et al., SLURP, EMNLP 2020: https://aclanthology.org/2020.emnlp-main.588/. Kim and Rush, Sequence-Level Knowledge Distillation, EMNLP 2016: https://aclanthology.org/D16-1139/. EleutherAI Pythia-70M model card: https://huggingface.co/EleutherAI/pythia-70m. k3d: https://k3d.io/; Kubernetes: https://kubernetes.io/docs/.